

examples in this chapter, I downloaded a file of Shakespeare's complete works, available for free online at <http://www.gutenberg.org/files/100/100-0.txt>, and then removed the Project Gutenberg boilerplate text on the top. You could also use the complete works of Mark Twain, available at <http://www.gutenberg.org/cache/epub/3200/pg3200.txt>. Read a corpus into Python as follows:

```
import requests
file = requests.get('http://www.bradfordtuckfield.com/shakespeare.txt')
file = file.text
text = file.replace('\n', '')
```

Here, we used the `requests` module to directly read a text file containing the collected works of Shakespeare from a website where it's being hosted, and then read it into our Python session in a variable called `text`.

After reading in your chosen corpus, rerun the code that created the `grams` variable. Here it is with the new definition of the `text` variable:

```
token = nltk.word_tokenize(text)
bigrams = ngrams(token, 2)
trigrams = ngrams(token, 3)
fourgrams = ngrams(token, 4)
fivegrams = ngrams(token, 5)
grams = [ngrams(token, 2), ngrams(token, 3), ngrams(token, 4), ngrams(token, 5)]
```

Our Strategy

Our strategy for generating search suggestions is simple. When a user types in a search, we check how many words are in their search. In other words, a user enters an n -gram and we determine what n is. When a user searches for an n -gram, we are helping them add to their search, so we will want to suggest an $n + 1$ -gram. We'll search our corpus and find all $n + 1$ -grams whose first n elements match our n -gram. For example, a user might search for `crane`, a 1-gram, and our corpus might contain the 2-grams `crane feather`, `crane operator`, and `crane neck`. Each is a potential search suggestion we could offer.

We could stop there, providing every $n + 1$ -gram whose first n elements matched the $n + 1$ -gram the user had entered. However, not all suggestions are equally good. For example, if we are working for a custom engine that